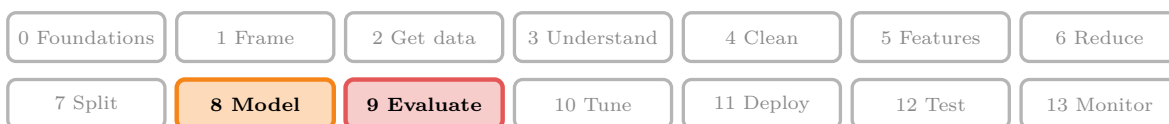


Stacking and Blending

Contents

1. Overview	2
2. From voting to stacking	3
2.1 One training row for the meta-model	3
3. The basic recipe in three steps	3
4. Stacking compared with bagging and boosting	4
5. The overfitting problem	4
6. Blending: a hold-out set	5
7. Stacking: K-fold out-of-fold predictions	6
8. Stacking in several layers	7
9. StackingClassifier in scikit-learn	8
9.1 The heart-disease data	9
9.2 The stack	9
9.3 Comparing the options	10
10. Summary	12
11. Key terms	12

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Ensemble learning ([Note 101](#)).
- **Compare with:** Train-test split ([Note 13](#)); Voting ensembles ([Note 104](#)); OOB score ([Note 113](#)); Boosting ([Note 119](#)).

1. Overview

Key point: Stacking trains several different models, turns their predictions into the input columns of a new dataset, and trains one more model, the meta-model, on it. To keep the meta-model honest, it must learn from predictions on rows the base models never saw: a hold-out set (blending) or K-fold out-of-fold predictions (stacking).

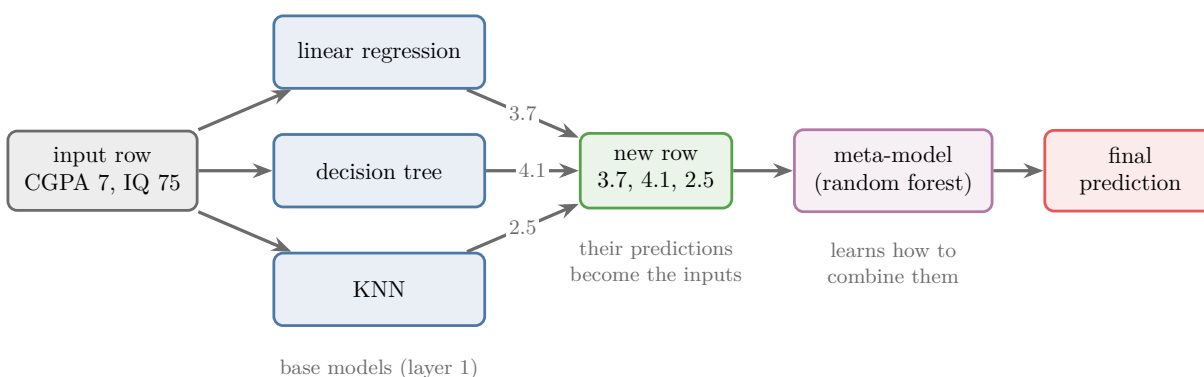


Figure 1: Stacking: three base models predict, their predictions form a new row, and the meta-model turns that row into the final prediction

Stacking was previewed in the [ensemble learning Note](#), section 4.2, as voting plus one more model on top. Figure 1 shows the full picture. This Note covers:

- how the new dataset for the meta-model is built;
- why predicting on the training rows ruins it, and the two fixes, blending and K-fold stacking;
- stacking in several layers;
- `StackingClassifier` in scikit-learn, on real heart-disease data.

2. From voting to stacking

Key point: Voting combines the base models with a fixed rule (majority or mean); stacking replaces the rule with a model that learns how to combine them.

In a voting ensemble ([voting ensemble Note](#), [voting regressor Note](#)) we train different algorithms on the same data and combine their outputs by a fixed rule. For a regression problem, three models predicting 3.5, 4.1 and 2.7 lakh rupees for one student give their mean, 3.43.

Stacking keeps the first half. Instead of averaging, it treats the three outputs as **inputs** for one more machine learning model, the **meta-model** (defined in the ensemble learning Note). The meta-model learns from data how much to believe each base model, and in what combination.

2.1 One training row for the meta-model

Key point: A student with CGPA 7, IQ 75 and package 3.5 becomes the row (3.7, 4.1, 2.5) with the same target 3.5.

Take one student from the training data: CGPA 7, IQ 75, package 3.5 lakh per annum. The three trained base models predict 3.7, 4.1 and 2.5 for this student.

For the meta-model this student becomes a new row: three input columns holding 3.7, 4.1 and 2.5, and the same target, 3.5. Every student gives one such row. The CGPA and IQ themselves are not passed on (section 8 shows the option that does pass them).

3. The basic recipe in three steps

Key point: Train the base models; let them predict every training row; train the meta-model on those predictions. To predict, run the base models first and the meta-model on their outputs.

Take a dataset of 1,000 students with CGPA, IQ and package, and three base models: linear regression, a decision tree and KNN. The meta-model is a random forest regressor.

1. **Train the base models.** Fit linear regression, the decision tree and KNN on the data, one by one.
2. **Build the new dataset.** Each base model predicts all 1,000 students. The predictions become three columns, `LR_pred`, `DT_pred` and `KNN_pred`; the package column stays the target. The new dataset has 1,000 rows and 4 columns.
3. **Train the meta-model** on the new dataset.

Prediction for a new student (say CGPA 7.7, IQ 100) follows the same path as Figure 1. The three base models predict first; their three numbers go to the meta-model, whose output is the answer.

Classification works the same way. The base models output classes (1 or 0) or probabilities, and the meta-model is a classifier.

4. Stacking compared with bagging and boosting

Key point: Stacking mixes different algorithms, and it combines them with a trained model instead of a vote, a mean or a weighted sum.

	Bagging and boosting	Stacking
Base models	one algorithm, many copies	different algorithms
Combining	majority vote or mean (bagging); weighted sum (boosting)	a meta-model trained on the base models' outputs

Bagging is covered in the [bagging intuition Note](#), boosting in the [bagging versus boosting Note](#).

5. The overfitting problem

Key point: In step 2 the base models predict the very rows they were trained on. Those predictions look far better than the base models really are, so the meta-model learns to trust the wrong models.

In the basic recipe, the base models are trained on the 1,000 students and then asked to predict the same 1,000 students. A model that overfits, such as a deep decision tree, reproduces its training targets almost exactly. Its column in the new dataset is then nearly a copy of the target.

The meta-model sees that column and decides the tree is almost always right. On new students the tree is much worse, but the meta-model still trusts it. The overfitting of one base model is passed up to the meta-model, and the whole stack fails.

The Notebook (section 7) measures this on the heart data. On their own training rows, the random forest and gradient boosting are off by only 0.09 and 0.07 on average (the gap between predicted probability and true class); on rows they did not see, by 0.30 and 0.25. The meta-model trained on the in-sample predictions gives those two models weights of 3.5 and 3.8, and KNN only 1.0.

There are two ways out. Both make sure the meta-model only learns from predictions on rows the base models did not train on:

- **blending:** a hold-out set;
- **stacking** in the strict sense: K-fold cross-validation.

Many people call both of them stacking. The difference is worth knowing, as it is a common interview question.

6. Blending: a hold-out set

Key point: Split the training data again. The base models learn on one part; the meta-model learns from their predictions on the other part.

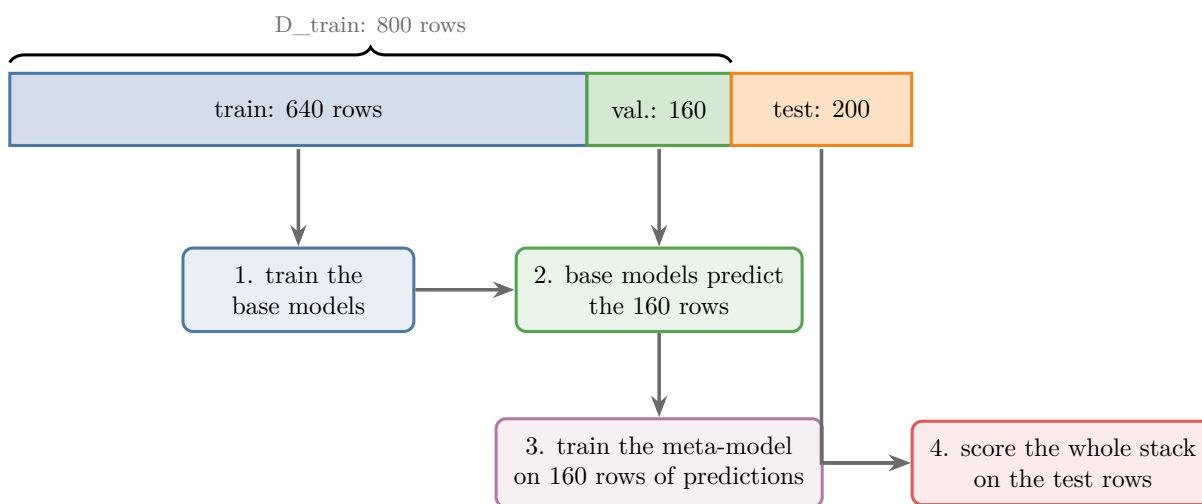


Figure 2: Blending: 1,000 rows split into 800 for training and 200 for testing; the 800 are split again into 640 for the base models and 160 for the meta-model

Blending builds the meta-model's dataset from a **hold-out set**: rows set aside before the base models are trained. With 1,000 students and the same architecture as section 3 (Figure 2):

1. **Split twice.** First 80/20 into D_train (800 rows) and D_test (200 rows). Then D_train again 80/20, into a smaller training set (640 rows) and a **validation set** (160 rows). All splits are random.
2. **Train the base models** on the 640 rows.
3. **Predict the validation set.** Each base model predicts the 160 validation rows. The three columns of predictions plus the true package form a new dataset of 160 rows and 4 columns.
4. **Train the meta-model** on these 160 rows.
5. **Evaluate.** Send the 200 test rows through the whole stack (base models, then meta-model) and compute the R^2 score ([regression metrics Note](#)).

The base models never saw the validation rows, so their predictions there are honest. The cost: the base models learn from only 640 rows, and the meta-model from only 160.

7. Stacking: K-fold out-of-fold predictions

Key point: Cut `D_train` into K folds. Each fold is predicted by a copy of the base model trained on the other $K-1$ folds, so every training row gets an honest prediction. Then the base models are retrained on all of `D_train`.

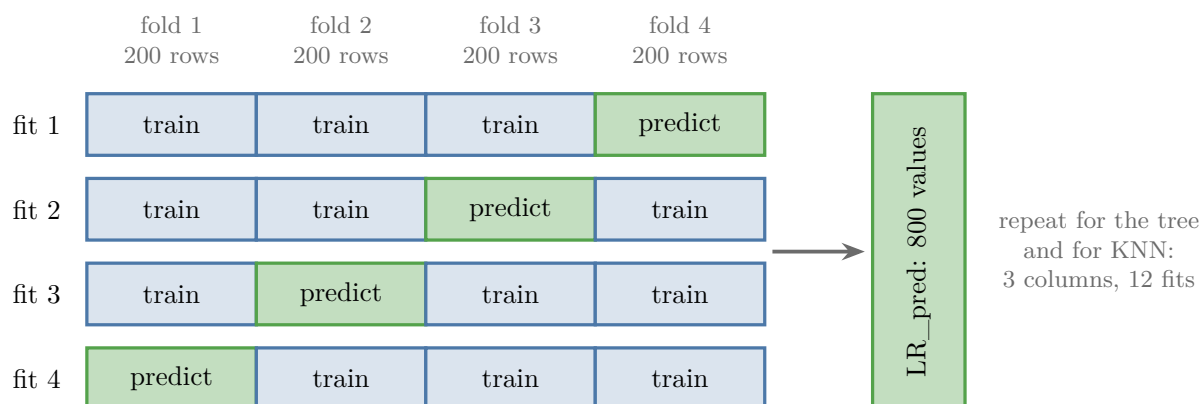


Figure 3: K-fold stacking with $K = 4$: each fit trains on three folds and predicts the fourth; the four predicted folds together fill one column of 800 out-of-fold predictions

Stacking in the strict sense uses the idea of K-fold cross-validation ([pipelines Note](#), section 8). We keep the 800 rows of `D_train` and the 200 test rows of section 6, and take $K = 4$ (5 or 10 are more common), so each fold has 200 rows.

1. **Out-of-fold predictions for the first base model** (Figure 3). Train linear regression on folds 1, 2 and 3 (600 rows) and predict fold 4. Train a new linear regression on folds 1, 2 and 4 and predict fold 3. Do the same for fold 2 and fold 1. After four fits every one of the 800 rows has a prediction from a model that did not see it: the column `LR_pred`. These are called **out-of-fold predictions**.
2. **Repeat for the other base models**. The same four fits for the decision tree give `DT_pred`, and for KNN give `KNN_pred`. That is 12 fits in all, and each base algorithm was trained 4 times.
3. **Train the meta-model** on the new dataset: 800 rows, the three prediction columns plus the true package.
4. **Retrain the base models on all of `D_train`**. Which of the 4 linear regressions do we keep? None of them. We train linear regression, the decision tree and KNN once more on all 800 rows, and these are the base models used for prediction.
5. **Evaluate** the stack on the 200 test rows, as in blending.

Note the order: the meta-model is trained first (step 3), the final base models afterwards (step 4).

Compared with blending, nothing is thrown away: every row of `D_train` trains the final base models, and every row also gives the meta-model one example. The price is K times more

training. This is the approach most libraries implement.

Extra: The meta-model learns from predictions of models trained on 3/4 of the data, but at prediction time it receives predictions of models trained on all of it. Those are slightly better, so the meta-model's inputs are a little more reliable than the ones it learned from. In practice this mismatch is small, and it is the reason a larger K (5 or 10) is usually preferred.

8. Stacking in several layers

Key point: The meta-model's inputs can themselves come from a second layer of models. With blending, every extra layer needs its own hold-out set.

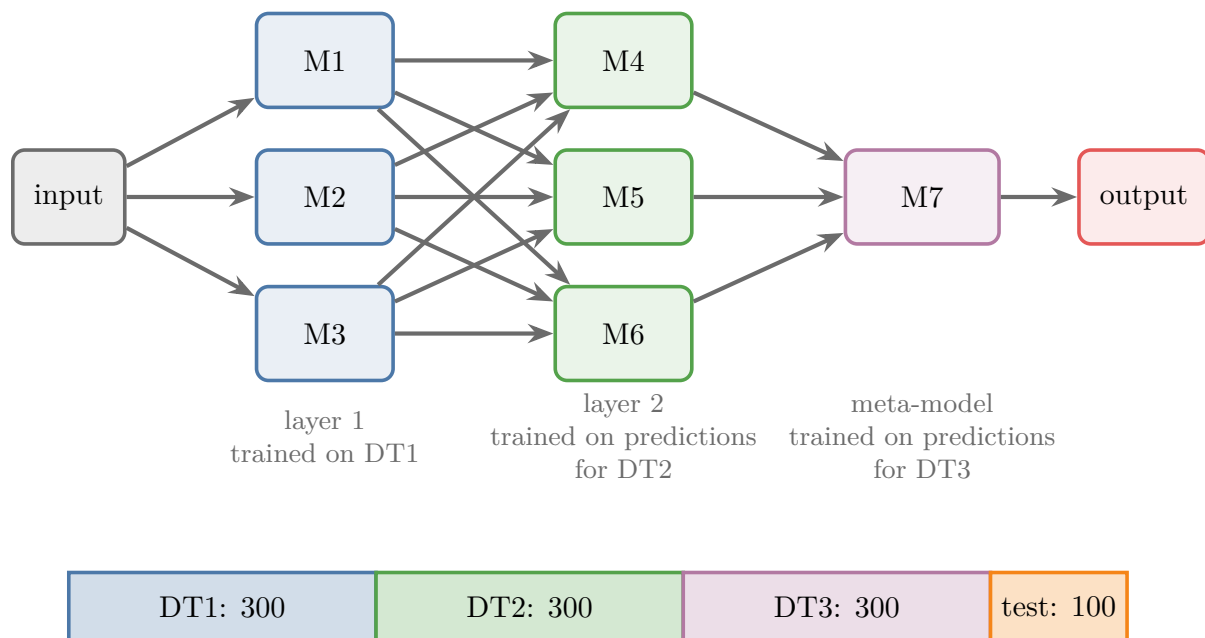


Figure 4: Two layers of base models and a meta-model, trained with blending on three hold-out sets of 300 rows

Nothing forces a single layer of base models. In Figure 4, three models M1, M2, M3 form layer 1, three more models M4, M5, M6 form layer 2, and M7 is the meta-model. Every model of layer 1 feeds every model of layer 2: the layers are **fully connected**. Any algorithm can sit anywhere: linear regression, KNN, a decision tree, gradient boosting, XGBoost, a random forest, even a neural network.

With blending, 1,000 students are split into 900 for training and 100 for testing, and the 900 into three sets of 300 (DT1, DT2, DT3):

1. **Layer 1.** Train M1, M2 and M3 on DT1. They predict DT2, giving a dataset of 300 rows with 3 prediction columns and the package.

2. **Layer 2.** Train M4, M5 and M6 on that dataset. To build their predictions on new rows, DT3 first goes through layer 1, then layer 2, giving another 300 rows with 3 columns and the package.
3. **Meta-model.** Train M7 on that last dataset.

Each layer needs fresh rows that the layer below has not seen, so every extra layer costs one more hold-out set. Doing the same with K-fold out-of-fold predictions is possible, but has to be coded by hand.

Deep stacks like this are common among the winning solutions of Kaggle competitions. One well-known winning solution used 64 models in the first layer, 15 in the second, 2 in the third and 1 meta-model at the top.

9. StackingClassifier in scikit-learn

Key point: `StackingClassifier(estimators, final_estimator, cv)` does K-fold stacking for us: out-of-fold predictions, the meta-model, and base models refitted on all the training data.

scikit-learn has `StackingClassifier` and `StackingRegressor` (since version 0.22) in `sklearn.ensemble`. Their main parameters:

Parameter	What it sets
<code>estimators</code>	the base models, as a list of (name, model) pairs
<code>final_estimator</code>	the meta-model (default: logistic regression for the classifier, ridge regression with built-in cross-validation for the regressor)
<code>cv</code>	the number of folds K for the out-of-fold predictions (default 5)
<code>stack_method</code>	classifier only: which output of each base model is passed on, "auto", "predict_proba", "decision_function" or "predict"
<code>passthrough</code>	False (default): the meta-model sees only the predictions; True : it also sees the original input columns
<code>n_jobs</code>	how many processor cores fit the base models in parallel

With `stack_method="auto"`, each base model passes its probabilities if it has `predict_proba`, otherwise its `decision_function`, otherwise its class predictions. For two classes only the probability of class 1 is kept, so each base model gives one column.

With `passthrough=True`, the meta-model's input row would hold the three predictions plus the CGPA and the IQ of section 2.1. This is used less often.

9.1 The heart-disease data

Key point: 303 patients, 13 inputs, a target of 1 (heart disease) or 0 (none); 242 patients train the models and 61 test them.

The data has 303 patients with 13 medical inputs (age, sex, chest-pain type, resting blood pressure, cholesterol and so on). The target is 1 for 165 patients with heart disease and 0 for 138 without. We split it 80/20 with `random_state=8`: 242 patients for training and 61 for testing.

9.2 The stack

Key point: Random forest, KNN and gradient boosting as base models, logistic regression as the meta-model, 10 folds: 0.869 test accuracy, above every base model alone.

Python: A three-model stack with a logistic regression on top.

```
from sklearn.ensemble import (StackingClassifier,
                               RandomForestClassifier, GradientBoostingClassifier)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression

estimators = [
    ("rf", RandomForestClassifier(n_estimators=10,
                                random_state=42)),
    ("knn", KNeighborsClassifier(n_neighbors=10)),
    ("gbdt", GradientBoostingClassifier(random_state=42)),
]
clf = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000),
    cv=10,
)
clf.fit(X_train, y_train)
clf.score(X_test, y_test)          # 0.869
```

The base models are listed as (name, model) pairs; `fit` runs the 10-fold out-of-fold predictions, trains the logistic regression on them and refits the three base models on all 242 rows. `score` returns the accuracy.

Alone, the three base models reach 0.787 (random forest), 0.639 (KNN) and 0.852 (gradient boosting) on the 61 test patients. The stack reaches **0.869**.

The meta-model's weights show whom it trusts: 2.31 for the random forest's probability, 1.75 for gradient boosting and 0.59 for KNN. It has learned that the unscaled KNN is the weakest

of the three.

Extra: KNN measures distances, and here cholesterol (around 250) dwarfs columns such as `sex` (0 or 1). Putting KNN in a pipeline with `StandardScaler` ([standardization Note](#)) lifts it from 0.639 to 0.836 alone. The Notebook uses the scaled KNN from section 4 onwards.

9.3 Comparing the options

Key point: On 61 test patients every method lands between 0.82 and 0.89; one patient is worth 1.6 points, so the differences are a few patients.

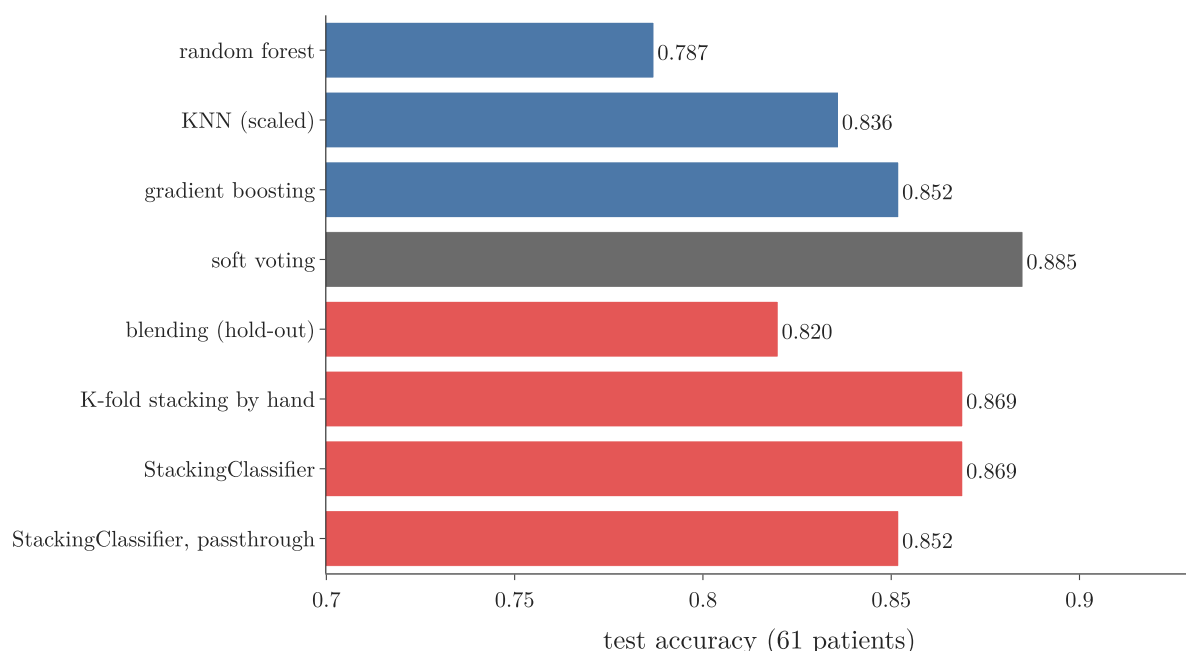


Figure 5: Test accuracy of the base models alone, soft voting and four kinds of stacking on the heart data (KNN scaled)

Figure 5 compares, with the scaled KNN:

- **soft voting** (averaging the three probabilities, [voting classifier Note](#)): 0.885;
- **blending by hand** (193 rows for the base models, 49 for the meta-model): 0.820;
- **K-fold stacking by hand** (4 folds, `cross_val_predict`): 0.869;
- **StackingClassifier** (10 folds): 0.869;
- **StackingClassifier with `passthrough=True`** (16 inputs to the meta-model): 0.852.

Blending is the weakest here: with only 242 training rows, holding 49 out hurts both the base models and the meta-model. On a test set this small, soft voting beating stacking by one patient says little. Stacking pays off most with many rows and base models that are good in different places.

Python: Out-of-fold predictions by hand.

```
from sklearn.model_selection import KFold, cross_val_predict
```

```
kf = KFold(n_splits=4, shuffle=True, random_state=42)
```

```
oof = np.column_stack([
    cross_val_predict(m, X_train, y_train, cv=kf,
                      method="predict_proba")[:, 1]
    for _, m in estimators
```

```
])
```

```
meta = LogisticRegression().fit(oof, y_train)
```

`cross_val_predict` returns, for every training row, the prediction of the copy of the model that did not train on that row's fold: exactly the column of Figure 3. `[:, 1]` keeps the probability of class 1.

10. Summary

	Basic recipe	Blending	Stacking (K-fold)
Meta-model learns from	predictions on the training rows	predictions on a hold-out validation set	out-of-fold predictions on all training rows
Honest inputs for the meta-model	no: overfit models look perfect	yes	yes
Rows used by the base models	all	only the training part	all (final refit)
Training cost	one fit per base model	one fit per base model	K + 1 fits per base model
Heart data, test accuracy	0.852	0.820	0.869

- Stacking feeds the base models' predictions to a meta-model, which learns how to combine them.
- Unlike bagging and boosting, the base models are different algorithms, and the combiner is trained, not a fixed rule.
- Predicting the training rows passes the base models' overfitting to the meta-model; a hold-out set (blending) or K-fold out-of-fold predictions (stacking) fix this.
- In K-fold stacking the meta-model is trained first, then the base models are refitted on all of `D_train`.
- Stacks can have several layers; with blending, each layer needs its own hold-out set.
- `StackingClassifier` and `StackingRegressor` take `estimators`, `final_estimator`, `cv`, `stack_method` (classifier) and `passthrough`.

11. Key terms

Term	Meaning
Stacking	An ensemble whose meta-model is trained on the base models' predictions; in the strict sense, with K-fold out-of-fold predictions
Blending	Stacking in which the meta-model is trained on the base models' predictions for a hold-out validation set
Hold-out set	Rows set aside before training, used only to produce honest predictions or scores
Validation set	The hold-out part of the training data in blending, on which the meta-model is trained
Out-of-fold prediction	A prediction for a row made by a model trained on the other folds, never on that row

Term	Meaning
Multi-layer stacking	Stacking with more than one layer of base models below the meta-model
Passthrough	Giving the meta-model the original input columns as well as the base models' predictions